LangChain Docs

Comience > Pensando en LangGraph

Comience

Pensando en LangGraph

Copiar página

Al crear un agente con LangGraph, primero lo dividirás en pasos discretos llamados **nodos** . Luego, describirás las diferentes decisiones y transiciones de cada nodo. Finalmente, conectarás los nodos entre sí mediante un **estado** compartido al que cada nodo puede acceder para leer y escribir.

En este tutorial, te guiaremos a través del proceso de pensamiento para crear un agente de correo electrónico de atención al cliente con LangGraph.

Empieza por el proceso que quieres automatizar.

Imagina que necesitas crear un agente de IA que gestione los correos electrónicos de atención al cliente. Tu equipo de producto te ha proporcionado los siguientes requisitos:

```
The agent should:

- Read incoming customer emails
- Classify them by urgency and topic
- Search relevant documentation to answer questions
- Draft appropriate responses
- Escalate complex issues to human agents
- Schedule follow-ups when needed

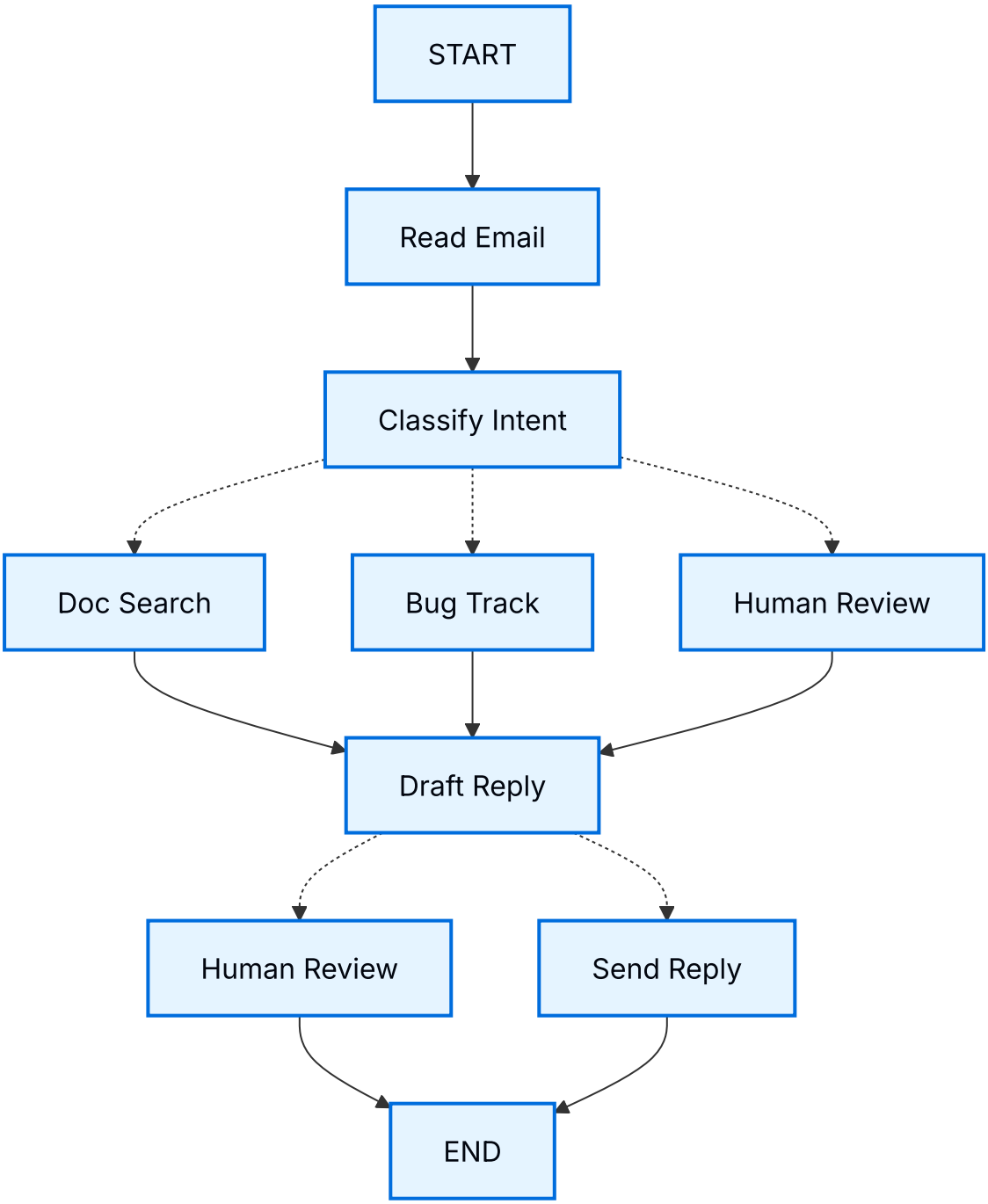
Example scenarios to handle:

1. Simple product question: "How do I reset my password?"
2. Bug report: "The export feature crashes when I select PDF format"
3. Urgent billing issue: "I was charged twice for my subscription!"
4. Feature request: "Can you add dark mode to the mobile app?"
5. Complex technical issue: "Our API integration fails intermittently with 504
```

LangChain Docs

Paso 1: Planifica tu flujo de trabajo como pasos discretos

Comienza por identificar los pasos distintivos de tu proceso. Cada paso se convertirá en un **nodo** (una función que realiza una acción específica). Luego, dibuja cómo se conectan estos pasos entre sí.



Las flechas de este diagrama muestran posibles rutas, pero la decisión final sobre qué ruta tomar se toma dentro de cada nodo.

LangChain Docs

- `Classify Intent` Utilice un LLM para categorizar la urgencia y el tema, y luego rediríjalo a la acción apropiada.
- `Doc Search` Consulta tu base de conocimientos para obtener información relevante.
- `Bug Track` : Crear o actualizar incidencia en el sistema de seguimiento
- `Draft Reply` Generar una respuesta apropiada
- `Human Review` : Remitir a un agente humano para su aprobación o gestión.
- `Send Reply` : Enviar la respuesta por correo electrónico

Observe que algunos nodos toman decisiones sobre adónde ir a continuación (`Classify Intent` , `Draft Reply` , `Human Review`), mientras que otros siempre proceden al mismo siguiente paso (`Read Email` siempre va a `Classify Intent` , `Doc Search` siempre va a `Draft Reply`).

Paso 2: Identificar qué debe hacer cada paso.

Para cada nodo de tu gráfico, determina qué tipo de operación representa y qué contexto necesita para funcionar correctamente.

Pasos del LLM
Úselo cuando necesite comprender, analizar, generar texto o tomar decisiones de razonamiento.

Pasos de datos
Úselo cuando necesite recuperar información de fuentes externas.

Pasos a seguir
Úselo cuando necesite realizar acciones externas.

Pasos de entrada del usuario
Úselo cuando necesite intervención humana.

- Clasificar intención
- Borrador de respuesta

Data steps

When a step needs to retrieve information from external sources:

- Document search
- Customer history lookup

Action steps

When a step needs to perform an external action:

- Send reply
- Bug track

User input steps

When a step needs human intervention:

- Human review node

Step 3: Design your state

State is the shared memory accessible to all nodes in your agent. Think of it as the notebook your agent uses to keep track of everything it learns and decides as it works through the process.

LangChain Docs

Include in state

Does it need to persist across steps? If yes, it goes in state.

Don't store

Can you derive it from other data? If yes, compute it when needed instead of storing it in state.

For our email agent, we need to track:

- The original email and sender info (can't reconstruct these later)
- Classification results (needed by multiple later/downstream nodes)
- Search results and customer data (expensive to re-fetch)
- The draft response (needs to persist through review)
- Execution metadata (for debugging and recovery)

Keep state raw, format prompts on-demand

A key principle: your state should store raw data, not formatted text. Format prompts inside nodes when you need them.

This separation means:

- Different nodes can format the same data differently for their needs
- You can change prompt templates without modifying your state schema
- Debugging is clearer—you see exactly what data each node received
- Your agent can evolve without breaking existing state

Let's define our state:

LangChain Docs

```
    intent: Literal["question", "bug", "billing", "feature", "complex"]
    urgency: Literal["low", "medium", "high", "critical"]
    topic: str
    summary: str

class EmailAgentState(TypedDict):
    # Raw email data
    email_content: str
    sender_email: str
    email_id: str

    # Classification result
    classification: EmailClassification | None

    # Raw search/API results
    search_results: list[str] | None  # List of raw document chunks
    customer_history: dict | None  # Raw customer data from CRM

    # Generated content
    draft_response: str | None
    messages: list[str] | None
```

Notice that the state contains only raw data—no prompt templates, no formatted strings, no instructions. The classification output is stored as a single dictionary, straight from the LLM.

Step 4: Build your nodes

Now we implement each step as a function. A node in LangGraph is just a Python function that takes the current state and returns updates to it.

Handle errors appropriately

Different errors need different handling strategies:

LangChain Docs

LLM-recoverable errors (tool failures, parsing issues)	LLM	Store error in state and loop back	LLM can see the error and adjust its approach
User-fixable errors (missing information, unclear instructions)	Human	Pause with `interrupt()`	Need user input to proceed
Recoverable failure after retries	Developer (declarative)	`error_handler`	Run a compensation/recovery branch after retry exhaustion
Unexpected errors	Developer	Let them bubble up	Unknown issues that need debugging

Transient errors | LLM-recoverable | User-fixable | Unexpected | Saga / compe

Add a retry policy to automatically retry network issues and rate limits.

Combine with `timeout=` to cap each attempt. See Fault tolerance for the full lifecycle.

```
from langgraph.types import RetryPolicy

workflow.add_node(
    "search_documentation",
    search_documentation,
    retry_policy=RetryPolicy(max_attempts=3, initial_interval=1.0)
)
```

Implementing our email agent nodes

We'll implement each node as a simple function. Remember: nodes take state, do work, and return updates.

LangChain Docs

▶ Response nodes

Step 5: Wire it together

Now we connect our nodes into a working graph. Since our nodes handle their own routing decisions, we only need a few essential edges.

To enable human-in-the-loop with `interrupt()`, we need to compile with a checkpointer to save state between runs:

▼ Graph compilation code

```python
# Create the graph
workflow = StateGraph(EmailAgentState)

# Add nodes with appropriate error handling
workflow.add_node("read_email", read_email)
workflow.add_node("classify_intent", classify_intent)

# Add retry policy for nodes that might have transient failures
workflow.add_node(
    "search_documentation",
    search_documentation,
    retry_policy=RetryPolicy(max_attempts=3)
)
workflow.add_node("bug_tracking", bug_tracking)
workflow.add_node("draft_response", draft_response)
workflow.add_node("human_review", human_review)
workflow.add_node("send_reply", send_reply)

# Add only the essential edges
workflow.add_edge(START, "read_email")
workflow.add_edge("read_email", "classify_intent")
workflow.add_edge("send_reply", END)

# Compile with checkpointer for persistence, in case run graph with Local_
memory = MemorySaver()
app = workflow.compile(checkpointer=memory)
```

The graph structure is minimal because routing happens inside nodes through `Command` objects. Each node declares where it can go using type hints like `Command[Literal["node1", "node2"]]`, making the flow explicit and traceable.

Try out your agent

Let's run our agent with an urgent billing issue that needs human review:

waits. It can resume days later, picking up exactly where it left off. The `thread_id` ensures all state for this conversation is preserved together.

Summary and next steps

Key Insights

Building this email agent has shown us the LangGraph way of thinking:

Break into discrete steps

Each node does one thing well. This decomposition enables streaming progress updates, durable execution that can pause and resume, and clear debugging since you can inspect state between steps.

State is shared memory

Store raw data, not formatted text. This lets different nodes use the same information in different ways.

Nodes are functions

They take state, do work, and return updates. When they need to make routing decisions, they specify both the state updates and the next destination.

Errors are part of the flow

Transient failures get retries, LLM-recoverable errors loop back with context, user-fixable problems pause for input, and unexpected errors bubble up for debugging.

Human input is first-class

The `interrupt()` function pauses execution indefinitely, saves all state, and resumes exactly where it left off

Graph structure emerges naturally

You define the essential connections, and your nodes handle their own routing logic. This keeps

LangChain Docs

Advanced considerations

- Node granularity trade-offs

¿Qué hacer ahora?

Esta fue una introducción al concepto de creación de agentes con LangGraph. Puedes ampliar esta base con:

patrones de intervención humana
Aprenda cómo agregar la aprobación de herramientas antes de la ejecución, la aprobación por lotes y otros patrones.

Subgrafos
Cree subgrafos para operaciones complejas de varios pasos.

Transmisión
Agregue la transmisión en directo para mostrar el progreso en tiempo real a los usuarios.

Observabilidad
Agregue observabilidad con LangSmith para depuración y monitorización.

Integración de herramientas
Integrar más herramientas para búsquedas web, consultas a bases de datos y llamadas a API.

Lógica de reintento
Implementar lógica de reintento con retroceso exponencial para operaciones fallidas.

LangChain Docs

Edita esta página en GitHub o crea una incidencia .

¿Te ha resultado útil esta página? Sí No

LangChain Docs

Recursos

Foro

Registro de cambios

Academia LangChain

Contactar con el departamento de ventas

Compañía

Hogar

Centro de confianza

Carreras profesionales

Blog